

Introduction to *Spring Boot*

This is where everything comes together. Spring Boot takes the Spring Framework concepts we learned (IoC, DI, Beans) and makes them incredibly easy to use — so you can build microservices with REST APIs in minutes, not hours.

- Prerequisites: Spring Framework concepts, REST API ▸ Concepts + code examples
- Level: Beginner

01 What is Spring Boot?

Spring Boot is a tool built **on top of the Spring Framework** that eliminates most of the setup and configuration work. It lets you create production-ready Spring applications with minimal effort.

Remember from the Spring Framework lesson: Spring is powerful but requires a lot of configuration. Spring Boot's philosophy is "**convention over configuration**" — it makes smart default choices for you, and you only override what you need to change.



Think of it like a smartphone

Spring Framework is like building a phone from individual components — screen, battery, processor, camera — and wiring them yourself. You have full control, but it takes a long time.

Spring Boot is the phone that comes pre-assembled out of the box. Turn it on, it just works. You can still customize the wallpaper, install apps, and change settings — but you don't have to build it from scratch.

🔗 **Big Picture:** Microservices (architecture) → REST API (communication rules) → Spring Framework (IoC, DI, Beans) → Spring Ecosystem (the map) → **Spring Boot (today)** — the tool that makes building microservices fast and easy.

02 What Problems Does Spring Boot Solve?

Before Spring Boot, starting a Spring project meant dealing with all of this manually:

PROBLEM	WITHOUT SPRING BOOT	WITH SPRING BOOT
Project setup	Create folder structure, configure build file, add dozens of dependencies manually	Use start.spring.io — generates everything in seconds
Dependency management	Find compatible versions of every library yourself	Starters bundle compatible dependencies automatically
Configuration	Write pages of XML or Java config for every feature	Auto-configuration — Spring Boot configures things based on what's on the classpath

PROBLEM	WITHOUT SPRING BOOT	WITH SPRING BOOT
Web server	Download, install, and configure Tomcat separately. Deploy WAR file.	Embedded Tomcat — built into your app. Just run the JAR.
Running the app	Package as WAR, deploy to external server	<code>java -jar myapp.jar</code> — that's it
Production readiness	Add health checks, metrics, and monitoring manually	Actuator — production endpoints out of the box

In short: Spring Boot removes the boring setup work so you can focus on writing business logic — the code that actually matters.

03 Key Features of Spring Boot

These are the **6 official features** listed on the Spring Boot page at <https://spring.io/projects/spring-boot>. Let's understand each one:

1. Create Stand-Alone Spring Applications

Your application is a **self-contained JAR file**. It doesn't need an external application server. Just run `java -jar product-service.jar` and it starts — the web server, the Spring Container, everything. This is what makes deploying microservices so simple: one JAR per service.

2. Embed Tomcat, Jetty, or Undertow Directly (No WAR Files)

The web server is **built into your application**, not the other way around. Before Spring Boot, you had to install Tomcat separately and deploy your app as a WAR file into it. Now the server is embedded — Tomcat by default, but you can switch to **Jetty** or **Undertow** by just changing a dependency. No installation, no external server to manage.

 **Microservices Connection:** This is why each microservice can run independently.

Product Service embeds its own Tomcat on port 8081. Order Service embeds its own Tomcat on port 8082. No shared server — each service is fully self-contained.

3. Provide Opinionated 'Starter' Dependencies

Instead of manually adding 10 compatible libraries, you add **one starter** that bundles everything you need. Spring Boot is "opinionated" — it makes smart default choices so you don't have to research which library versions work together.

STARTER	WHAT IT BUNDLES
<code>spring-boot-starter-web</code>	Spring MVC + embedded Tomcat + Jackson (JSON) + validation
<code>spring-boot-starter-data-jpa</code>	Spring Data JPA + Hibernate + JDBC
<code>spring-boot-starter-security</code>	Spring Security + authentication + authorization
<code>spring-boot-starter-test</code>	JUnit 5 + Mockito + Spring Test utilities
<code>spring-boot-starter-actuator</code>	Health checks + metrics + monitoring endpoints

4. Automatically Configure Spring and 3rd Party Libraries

This is the **auto-configuration magic**. Spring Boot looks at what's on your classpath (your dependencies) and configures things automatically:

- Added a MySQL driver? Spring Boot auto-configures a **DataSource**
- Added Spring Web starter? Spring Boot auto-configures an **embedded Tomcat** and **Spring MVC**
- Added Spring Data JPA? Spring Boot auto-configures **Hibernate** and **entity scanning**

→ Added Spring Security? Spring Boot auto-configures **authentication** with a default login page

You only write configuration when you need to **override a default**. For most cases, the defaults just work.

5. Provide Production-Ready Features

Spring Boot includes built-in features for running in production:

- **Health checks** — `/actuator/health` tells you if the app is running and if the database is connected
- **Metrics** — `/actuator/metrics` tracks request counts, response times, memory usage
- **Externalized configuration** — configure your app differently per environment (dev, staging, production) without changing code. Use `application.properties`, environment variables, or a config server.
- **Profiles** — switch between `application-dev.properties` and `application-prod.properties` with a single flag

 **Microservices Connection:** Externalized configuration is critical for microservices. The same JAR runs in dev (pointing to localhost database) and production (pointing to a real database) — only the properties change. In a full microservices setup, **Spring Cloud Config** centralizes configuration for all services in one place.

6. No Code Generation and No XML Configuration

Spring Boot doesn't generate code behind the scenes and doesn't require any XML files. Everything is done through:

- **Annotations** — `@RestController`, `@Service`, `@Entity`, etc.
- **application.properties** — simple key-value pairs
- **Java configuration** — if you need custom config, it's done in Java classes, not XML

What you see in your code is what runs. No hidden generated files, no XML surprises.

Source: These 6 features are listed on the official Spring Boot page at <https://spring.io/projects/spring-boot>

Bonus Features Worth Knowing



Actuator

The tool that provides the production-ready features mentioned above. Add `spring-boot-starter-actuator` and you get `/actuator/health` , `/actuator/info` , `/actuator/metrics` and more — essential for monitoring microservices.



Easy Testing

Built-in testing support.

`@SpringBootTest` spins up your entire application context for integration tests. Mocking, slicing, and test utilities are all included via the test starter.



DevTools

Auto-restart on code changes during development. Save a file and the app restarts in seconds. Add `spring-boot-devtools` and it just works — a huge time saver.

Every Spring Boot project starts at <https://start.spring.io> — the Spring Initializr. It's a web tool that generates a ready-to-run project for you.

What You Choose

SETTING	WHAT IT MEANS	TYPICAL CHOICE
Project	Build tool	Maven (most common) or Gradle
Language	Programming language	Java
Spring Boot version	Which version of Boot	Latest stable (not SNAPSHOT)
Group	Your organization's domain (reversed)	<code>com.example</code>
Artifact	Your project name	<code>product-service</code>
Packaging	How to package the app	JAR (not WAR)
Java version	Which JDK	17 or 21
Dependencies	What features you need	Pick from the list (see below)

Common Dependencies for Microservices

DEPENDENCY	WHAT IT ADDS	WHEN YOU NEED IT
<code>Spring Web</code>	Spring MVC, Tomcat, REST support	Always — every microservice serves HTTP
<code>Spring Data JPA</code>	JPA + Hibernate for database access	When your service uses a relational database

DEPENDENCY	WHAT IT ADDS	WHEN YOU NEED IT
MySQL Driver	MySQL JDBC driver	When using MySQL (swap for PostgreSQL, H2, etc.)
Spring Boot DevTools	Auto-restart on code changes	During development — saves time
Lombok	Reduces boilerplate (getters, setters, constructors)	Useful later — skip for now during training
Spring Boot Actuator	Health checks, metrics, monitoring	For production-ready services
Validation	Bean validation (input checking)	When you need to validate request data

Tip: For our learning purposes, start with just **Spring Web** and **Spring Data JPA** + a database driver. You can always add more dependencies later — Spring Boot makes that easy.

05 Project Structure

When you generate a project, Spring Initializr creates this folder structure. Every Spring Boot project follows the same layout:

PROJECT STRUCTURE

```
product-service/
├─ src/
│   ├─ main/
│   │   └─ java/
│   │       └─ com/example/productservice/
│   │           └─ ProductServiceApplication.java ← entry point
│   │           └─ controller/
│   │               └─ ProductController.java ← REST endpoints
```




Key files to know: The **Application class** is the entry point (has `main()`). The **controller/service/repository/model** folders organize your code by layer. The **application.properties** file configures everything. The **pom.xml** lists your dependencies.

06 Spring Boot Annotations Explained

Spring Boot uses **annotations** (the `@` tags above your classes and methods) to tell Spring what each class does. Here are the ones you'll use most:

The Entry Point

JAVA

```
@SpringBootApplication
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

`@SpringBootApplication` is a shortcut that combines three annotations:

- `@Configuration` — this class can define Beans
- `@EnableAutoConfiguration` — turn on Spring Boot's auto-configuration magic
- `@ComponentScan` — scan this package and sub-packages for Spring components

🔗 **Spring Framework Connection:** Remember the Spring Container (`ApplicationContext`) from the last lesson? `SpringApplication.run()` is what **starts that Container**. It creates all your Beans, wires the dependencies, and starts the embedded Tomcat server — all in one line.

The Stereotype Annotations

These tell Spring "this class is a Bean — please create and manage it."

ANNOTATION	USE ON	PURPOSE
<code>@Component</code>	Any class	Generic — "this is a Spring-managed Bean"
<code>@Controller</code> / <code>@RestController</code>	Controller classes	"This class handles HTTP requests"
<code>@Service</code>	Service classes	"This class contains business logic"
<code>@Repository</code>	Repository classes	"This class talks to the database"

They all do the same thing — make the class a Bean. The different names (`@Service`, `@Repository`, `@Controller`) are just for clarity and readability. When another developer sees `@Service`, they immediately know it's a business logic class. Spring also adds extra behavior to `@Repository` (exception translation) and `@Controller` (HTTP handling).

@Controller vs @RestController — What's the Difference?

This confuses many beginners because they look so similar. The key difference is **what the methods return**:

ASPECT	@CONTROLLER	@RESTCONTROLLER
Returns	A view name (HTML page) — Spring looks for a template file to render	Data directly (JSON/XML) — the return value IS the response body
Use case	Traditional web apps that serve HTML pages (Thymeleaf, JSP)	REST APIs that serve JSON data to clients or other microservices
Under the hood	@Controller only	@Controller + @ResponseBody combined into one annotation
To return JSON from @Controller	You must add @ResponseBody on every method individually	Not needed — every method automatically returns JSON



Think of it this way

@Controller = a waiter who takes your order and brings you a *plated meal* (a rendered HTML page).

@RestController = a waiter who takes your order and brings you the *raw ingredients in a box* (JSON data) — you decide how to present them yourself.

For microservices and REST APIs, you'll almost always use **@RestController** because you're sending JSON data, not HTML pages.

 **Microservices Connection:** Microservices communicate with JSON data, not HTML. That's why every controller in a microservice uses **@RestController**. The client (browser, mobile app, or another microservice) receives raw JSON and decides how to display it. This is the **client-server separation** we learned in the REST constraints.

REST Endpoint Annotations

ANNOTATION	PURPOSE	EXAMPLE
<code>@RestController</code>	Marks a class as a REST controller (returns JSON, not HTML)	On the class
<code>@RequestMapping</code>	Sets the base URL path for all endpoints in this controller	<code>@RequestMapping("/api/products")</code>
<code>@GetMapping</code>	Handles GET requests	<code>@GetMapping("/{id}")</code>
<code>@PostMapping</code>	Handles POST requests	<code>@PostMapping</code>
<code>@PutMapping</code>	Handles PUT requests	<code>@PutMapping("/{id}")</code>
<code>@DeleteMapping</code>	Handles DELETE requests	<code>@DeleteMapping("/{id}")</code>
<code>@PathVariable</code>	Extracts a value from the URL path	<code>@PathVariable Long id</code>
<code>@RequestParam</code>	Extracts a value from the query string (<code>? key=value</code>)	<code>@RequestParam String category</code>
<code>@RequestBody</code>	Reads JSON from the request body into a Java object	<code>@RequestBody Product product</code>

@PathVariable vs @RequestParam — When to Use Which?

Both extract values from the URL, but from **different parts**:

ASPECT	@PATHVARIABLE	@REQUESTPARAM
Reads from	The URL path	The query string (after ?)
URL example	<code>/api/products/42</code>	<code>/api/products?category=laptops</code>
Use for	Identifying a specific resource (by ID)	Filtering, sorting, pagination
Required?	Yes — part of the path	Optional by default (can set <code>required=false</code>)

Quick example showing both in action:

JAVA

```
// @PathVariable – get one product by ID
// GET /api/products/42
@GetMapping("/{id}")
public Product getById(@PathVariable Long id) { ... }

// @RequestParam – filter products by category
// GET /api/products?category=laptops&sort=price
@GetMapping
public List<Product> search(
    @RequestParam String category,
    @RequestParam(defaultValue = "name") String sort) { ... }
```

Rule of thumb: Use `@PathVariable` when the value identifies *which* resource (e.g., `/products/42`). Use `@RequestParam` when the value describes *how* to filter, sort, or paginate a collection (e.g., `?category=laptops&page=2`).

Dependency Injection Annotation

ANNOTATION

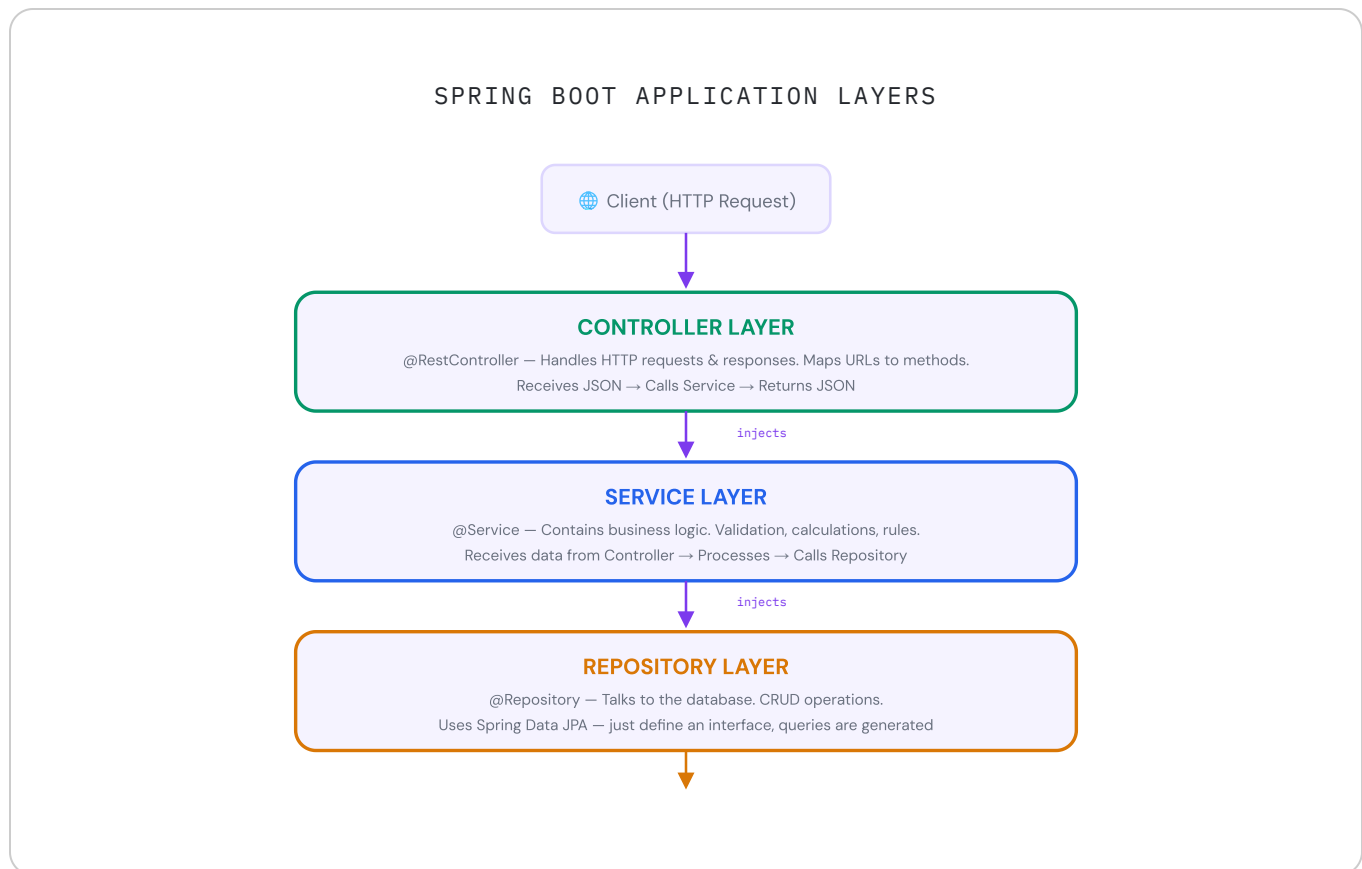
PURPOSE

`@Autowired`

Tells Spring to inject the dependency automatically. Used on constructors, setters, or fields. Constructor injection is recommended (and `@Autowired` is optional on constructors since Spring 4.3).

07 The Three Layers

Every Spring Boot microservice is organized into **three layers**. Each layer has a specific job, and they connect through Dependency Injection.



Model (Entity) — The Data Class

JAVA

```
@Entity // marks this as a database table
@Table(name = "products") // specifies the actual table name
```

```

public class Product {

    @Id                                     // primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)              // NOT NULL constraint
    private String name;

    private Double price;
    private String category;

    @Column(name = "in_stock")             // maps to a different column name
    private Boolean inStock;

    // constructors, getters, setters
}

```

JPA Annotations for the Model

ANNOTATION	PURPOSE	EXAMPLE
@Entity	Marks this class as a JPA entity (maps to a database table)	Required on the class
@Table	Specifies the table name . Without it, the table name defaults to the class name (<code>Product</code> → table <code>product</code>)	@Table(name = "products")
@Id	Marks the primary key field	Required — every entity needs one
@GeneratedValue	Auto-generate the ID (auto-increment)	strategy = GenerationType.IDENTITY

ANNOTATION	PURPOSE	EXAMPLE
<code>@Column</code>	Customize the column: name, nullable, length, unique	<code>@Column(name = "in_stock", nullable = false)</code>

When do you need @Table? If your class name matches your table name, you don't need it — Product automatically maps to a product table. Use @Table when the names differ, for example when your table is products (plural) or tbl_product (prefixed). Same logic applies to @Column — only needed when the field name differs from the column name, or when you want to add constraints like `nullable = false`.

Repository — Database Access

JAVA

```
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // That's it! Spring Data JPA generates these methods for you:
    // findAll(), findById(), save(), deleteById(), count(), etc.

    // You can add custom queries by just naming a method:
    List<Product> findByCategory(String category);
    List<Product> findByInStockTrue();
}
```

This is the magic of Spring Data JPA. You write an *interface* (not a class), extend JpaRepository, and Spring generates the full implementation at runtime. `findByCategory` automatically becomes a `SELECT * FROM product WHERE category = ?` query. No SQL needed.

Service — Business Logic


```
@Service
public class ProductService {

    private final ProductRepository productRepository;

    // Constructor Injection – Spring injects ProductRepository here
    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    public Product getProductById(Long id) {
        return productRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Product not found"));
    }

    public Product createProduct(Product product) {
        return productRepository.save(product);
    }

    public void deleteProduct(Long id) {
        productRepository.deleteById(id);
    }
}
```

 **Spring Framework Connection:** Notice there's no new `ProductRepository()` anywhere. The constructor receives it as a parameter — that's **Constructor Injection** (DI). Spring's Container creates the repository Bean and injects it automatically. This is IoC in action.

Controller — REST Endpoints

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public List<Product> getAllProducts() {
        return productService.getAllProducts();
    }

    @GetMapping("/{id}")
    public Product getProductById(@PathVariable Long id) {
        return productService.getProductById(id);
    }

    @PostMapping
    public Product createProduct(@RequestBody Product product) {
        return productService.createProduct(product);
    }

    @DeleteMapping("/{id}")
    public void deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
    }
}
```

 **REST API Connection:** Look at those annotations — @GetMapping, @PostMapping, @DeleteMapping, @PathVariable, @RequestBody. These map directly to the REST concepts we learned: HTTP methods, URL paths, request bodies, and JSON responses. Spring Boot turns REST theory into working code.

08 Configuration with application.properties

This single file controls everything about your application. It lives in `src/main/resources/application.properties` .

Basic Server Configuration

PROPERTIES

```
# Server
server.port=8081

# Application name (used by service discovery)
spring.application.name=product-service
```

Database Configuration

PROPERTIES

```
# MySQL connection
spring.datasource.url=jdbc:mysql://localhost:3306/products_db
spring.datasource.username=root
spring.datasource.password=secret
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Using H2 In-Memory Database (for testing/learning)

PROPERTIES

```
# H2 in-memory database (no installation needed)
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

```
# Enable H2 console (access at /h2-console)
spring.h2.console.enabled=true
```

Logging Configuration

PROPERTIES

```
# Logging level
logging.level.root=INFO
logging.level.com.example.productservice=DEBUG
logging.level.org.springframework.web=DEBUG
```

Common Properties Reference

PROPERTY	WHAT IT DOES	DEFAULT
<code>server.port</code>	Which port the app runs on	8080
<code>spring.application.name</code>	Name of your service (for discovery)	—
<code>spring.datasource.url</code>	Database connection URL	—
<code>spring.jpa.hibernate.ddl-auto</code>	Auto-create/update tables	none
<code>spring.jpa.show-sql</code>	Print SQL queries in the console	false
<code>logging.level.root</code>	Global logging level	INFO

About ddl-auto values: Use update for development (auto-creates/updates tables). Use validate or none in production — you don't want your app accidentally changing the database schema. Never use create-drop in production — it deletes everything on shutdown.

 **Microservices Connection:** Each microservice has its **own** application.properties with its own port, its own database, and its own name. Product Service runs on :8081 with products_db. Order Service runs on :8082 with orders_db. Completely independent — just like we learned in the microservices lesson.

 **Official documentation:** Full list of all available application properties:
<https://docs.spring.io/spring-boot/appendix/application-properties/index.html>

09 Starters in pom.xml

We covered the most common starters in Section 03. Here are a few more you'll encounter, plus how they look in your actual `pom.xml` file:

STARTER	WHAT IT INCLUDES
<code>spring-boot-starter-validation</code>	Bean Validation (Hibernate Validator) — validate request data
<code>spring-boot-starter-amqp</code>	Spring AMQP + RabbitMQ client — async messaging
<code>spring-boot-starter-mail</code>	Java Mail + Spring mail support — sending emails
<code>spring-boot-starter-cache</code>	Spring Cache abstraction — caching support

Example: pom.xml Dependencies

XML (POM.XML)

```
<dependencies>
  <!-- REST API support -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>

<!-- Database access -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- MySQL driver -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

<!-- Testing -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>
```

Notice: No version numbers on the Spring starters. Spring Boot's parent POM manages all versions for you — guaranteed to be compatible. This is one of the biggest time-savers.

 **Official documentation:** Full list of all available starters:

<https://docs.spring.io/spring-boot/reference/using/build-systems.html>

10 Running Your Application

Three Ways to Run

- 1 **From your IDE** — Right-click the Application class → Run. The fastest way during development.
- 2 **Using Maven** — Run `./mvnw spring-boot:run` from the terminal. No IDE needed.
- 3 **As a JAR** — Build with `./mvnw clean package`, then run with `java -jar target/product-service-0.0.1-SNAPSHOT.jar`. This is how you deploy to production.

What Happens When You Start

- 1 **Spring Boot starts** — reads `application.properties`, detects your dependencies
- 2 **Auto-configuration runs** — sees Spring Web? Configures Tomcat. Sees JPA? Configures Hibernate and DataSource.
- 3 **Component scan runs** — finds all classes with `@Component`, `@Service`, `@Repository`, `@Controller`
- 4 **Beans are created** — the Spring Container creates instances of all your components
- 5 **Dependencies are injected** — Controller gets Service, Service gets Repository (DI)
- 6 **Embedded Tomcat starts** — your REST API is now live on the configured port

CONSOLE OUTPUT

```

      ____          _          ____
  /\ /  ___ '  _ _ _ ( _ ) _ _ _ _ _ \ \ \ \
( ( ) \___ / ' / ' / / ' \ / _ ' / \ \ \ \
 \ /  ___ ) / _ ) / / / / / / ( / / ) ) ) )
```

```
' /____/ .__/_/ |_/_/ |_\\__, / / / / /
=====/_/=====|___/=/_/_/_/
```

Started ProductServiceApplication in 2.3 seconds
Tomcat started on port 8081

 **Microservices Connection:** In production, each microservice runs as its own JAR: `java -jar product-service.jar`, `java -jar order-service.jar`, `java -jar payment-service.jar`. Each starts its own Spring Container, its own Tomcat, on its own port. Completely independent processes — exactly the microservices architecture we learned in lesson 1.

11 ResponseEntity & @ResponseStatus

So far, our controller methods just return objects and Spring converts them to JSON with a 200 OK status. But in real REST APIs, you need to control the **HTTP status code** and sometimes the **headers** too. That's what `ResponseEntity` and `@ResponseStatus` are for.

@ResponseStatus — Simple Way

Annotate a method to always return a specific status code:

JAVA

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)    // always returns 201
public Product createProduct(@RequestBody Product product) {
    return productService.createProduct(product);
}

@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT) // always returns 204
public void deleteProduct(@PathVariable Long id) {
```



```
productService.deleteProduct(id);  
}
```

ResponseBody — Full Control

When you need to return **different status codes** depending on the situation, or add custom headers:

JAVA

```
@GetMapping("/{id}")  
public ResponseEntity<Product> getProductById(@PathVariable Long id) {  
    Product product = productService.getProductById(id);  
    if (product == null) {  
        return ResponseEntity.notFound().build();           // 404  
    }  
    return ResponseEntity.ok(product);                       // 200 + JSON body  
}  
  
@PostMapping  
public ResponseEntity<Product> createProduct(@RequestBody Product product) {  
    Product saved = productService.createProduct(product);  
    return ResponseEntity  
        .status(HttpStatus.CREATED)                          // 201  
        .body(saved);                                         // JSON body  
}
```

When to Use Which?

APPROACH	USE WHEN
<code>@ResponseStatus</code>	The method always returns the same status code. Simple and clean.
<code>ResponseBody</code>	The status code depends on logic (found vs not found, created vs conflict). Gives full control over status, headers, and body.

 **REST API Connection:** Remember the status codes we learned? 200 OK, 201 Created, 204 No Content, 404 Not Found — this is how you actually set them in Spring Boot. Proper status codes are critical for microservices because the calling service uses them to decide what to do next (retry, fallback, show error).

12 Exception Handling

When something goes wrong (product not found, invalid data, server error), you need to return a **clean error response** — not a stack trace. Spring Boot provides two ways to handle exceptions.

@ExceptionHandler — Per Controller

Handle a specific exception inside one controller:

JAVA

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    // ... your endpoints ...

    @ExceptionHandler(RuntimeException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, String> handleNotFound(RuntimeException ex) {
        return Map.of(
            "error", "NOT_FOUND",
            "message", ex.getMessage()
        );
    }
}
```

@ControllerAdvice — Global (All Controllers)

Handle exceptions across **all controllers** in one place. This is the recommended approach:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, Object> handleProductNotFound(ProductNotFoundException ex) {
        return Map.of(
            "error", "NOT_FOUND",
            "message", ex.getMessage(),
            "timestamp", LocalDateTime.now()
        );
    }

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public Map<String, Object> handleGeneral(Exception ex) {
        return Map.of(
            "error", "INTERNAL_ERROR",
            "message", "Something went wrong"
        );
    }
}
```

Why @RestControllerAdvice? It's @ControllerAdvice + @ResponseBody combined — same pattern as @RestController vs @Controller. It catches exceptions from all controllers and returns JSON error responses. One class handles all your error responses consistently.

 **Microservices Connection:** Consistent error responses are critical in microservices. When Order Service calls Product Service and gets a 404 with {"error": "NOT_FOUND", "message": "..."}, it can handle it cleanly. Random stack traces or inconsistent formats would break the calling service.

13 Profiles — Environment-Specific Configuration

Your application needs different settings for development (local database, debug logging) and production (real database, minimal logging). **Profiles** let you switch configurations without changing code.

How It Works

Create separate properties files per environment:

PROJECT STRUCTURE

```
src/main/resources/
├─ application.properties      ← shared settings (always loaded)
├─ application-dev.properties  ← development overrides
└─ application-prod.properties ← production overrides
```

APPLICATION.PROPERTIES (SHARED)

```
spring.application.name=product-service
server.port=8081
```

APPLICATION-DEV.PROPERTIES

```
# Development — H2 in-memory, debug logging
spring.datasource.url=jdbc:h2:mem:testdb
spring.jpa.show-sql=true
logging.level.com.example=DEBUG
```

APPLICATION-PROD.PROPERTIES

```
# Production — real MySQL, minimal logging
spring.datasource.url=jdbc:mysql://db-server:3306/products_db
spring.datasource.username=app_user
spring.datasource.password=${DB_PASSWORD}
spring.jpa.show-sql=false
logging.level.com.example=WARN
```

Activating a Profile

METHOD	HOW
In application.properties	<code>spring.profiles.active=dev</code>
Command line	<code>java -jar app.jar --spring.profiles.active=prod</code>
Environment variable	<code>SPRING_PROFILES_ACTIVE=prod</code>

 **Microservices Connection:** In a microservices setup, the same JAR runs in every environment — only the active profile changes. In Docker or Kubernetes, you set the profile via an environment variable. Combined with **Spring Cloud Config**, you can manage configuration for all services from one central server.

14 Lombok, Actuator & Testing — Quick Preview

These are tools you'll use in the hands-on session. Here's a quick preview so you know what to expect.

Lombok — Less Boilerplate Code

 **For this training, we will NOT use Lombok yet.** Write your getters, setters, and constructors manually. Why? Because you need to understand what these methods do and how they work before letting a tool generate them for you. Once you're comfortable with the fundamentals, Lombok will save you a lot of time — but not before you understand what's happening underneath.

That said, you should know what Lombok is because you'll see it in real projects and online tutorials. Java requires a lot of repetitive code: getters, setters, constructors, `toString()`. Lombok generates them at compile time using annotations:

```

@Data                                // generates getters, setters, toString, equals,
@NoArgsConstructor                    // generates empty constructor
@AllArgsConstructor                    // generates constructor with all fields
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private Double price;
    private String category;

    // No getters, setters, constructors needed - Lombok handles it!
}

```

LOMBOK ANNOTATION	WHAT IT GENERATES
@Getter / @Setter	Getter and/or setter methods for all fields
@ToString	toString() method
@EqualsAndHashCode	equals() and hashCode() methods
@Data	All of the above combined — the most commonly used
@NoArgsConstructor	Empty constructor — Product()
@AllArgsConstructor	Constructor with all fields — Product(id, name, price, ...)
@Builder	Builder pattern — Product.builder().name("...").price(9.99).build()

When will we use Lombok? After you're confident writing getters, setters, constructors, and `toString()` manually. At that point, Lombok becomes a productivity tool — not a crutch. Note: Lombok requires an IDE plugin (IntelliJ: install the Lombok plugin and enable annotation processing).

Actuator — Monitoring Your Application

Add `spring-boot-starter-actuator` and you get built-in endpoints for monitoring:

ENDPOINT	WHAT IT RETURNS
<code>/actuator/health</code>	Application health: <code>{"status": "UP"}</code> — includes database connectivity
<code>/actuator/info</code>	Application info (version, description — you configure this)
<code>/actuator/metrics</code>	List of available metrics (request counts, memory, CPU)
<code>/actuator/env</code>	Environment properties and configuration values
<code>/actuator/beans</code>	List of all Beans in the Spring Container

PROPERTIES

```
# Expose all actuator endpoints (default only exposes health)
management.endpoints.web.exposure.include=*
```

 **Microservices Connection:** When you have 10+ microservices running, `/actuator/health` is how your monitoring system (Prometheus, Kubernetes) knows which services are up and which are down. Service discovery tools like Eureka also use health endpoints to decide whether to route traffic to an instance.

Testing — @SpringBootTest

Spring Boot makes testing easy. `@SpringBootTest` starts the full application context so you can test your endpoints end-to-end:

JAVA

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class ProductControllerTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    void shouldGetAllProducts() {
        ResponseEntity<Product[]> response = restTemplate
            .getForEntity("/api/products", Product[].class);

        assertEquals(HttpStatus.OK, response.getStatusCode());
        assertNotNull(response.getBody());
    }
}
```

ANNOTATION	WHAT IT DOES
<code>@SpringBootTest</code>	Starts the full application context — tests everything together
<code>@WebMvcTest</code>	Tests only the controller layer (faster — no database, no service)
<code>@DataJpaTest</code>	Tests only the repository layer with an in-memory database
<code>@MockBean</code>	Replaces a real Bean with a mock (fake) for isolated testing

We'll do hands-on testing in the next session. For now, just know that Spring Boot has built-in support for different types of tests: full integration tests (`@SpringBootTest`), controller-only tests (`@WebMvcTest`), and database-only tests (`@DataJpaTest`).

15 Summary

Everything at a Glance

CONCEPT	WHAT IT IS
<code>@SpringBootApplication</code>	Entry point — starts the Container, auto-config, and component scan
<code>@RestController</code>	This class handles HTTP requests and returns JSON
<code>@Controller</code>	This class handles HTTP requests and returns HTML views
<code>@Service</code>	This class contains business logic
<code>@Repository</code>	This class talks to the database
<code>@Entity</code> / <code>@Table</code> / <code>@Column</code>	Map a Java class and its fields to a database table and columns
<code>@GetMapping</code> / <code>@PostMapping</code> / etc.	Map HTTP methods to Java methods
<code>@PathVariable</code>	Extract value from URL path (<code>/products/42</code>)
<code>@RequestParam</code>	Extract value from query string (<code>?category=laptops</code>)
<code>@RequestBody</code>	Read JSON from request body into Java object
<code>ResponseEntity</code>	Return custom status codes, headers, and body from controller methods
<code>@ResponseStatus</code>	Set a fixed HTTP status code on a method
<code>@RestControllerAdvice</code>	Global exception handler for all controllers — returns JSON errors

CONCEPT	WHAT IT IS
<code>@ExceptionHandler</code>	Catches a specific exception and returns a custom error response
Profiles	Environment-specific config: <code>application-dev.properties</code> vs <code>application-prod.properties</code>
Lombok <code>@Data</code>	Auto-generates getters, setters, constructors, toString — skip for now , learn manually first
Actuator	Built-in health checks, metrics, and monitoring endpoints
<code>@SpringBootTest</code>	Integration test annotation — starts the full application context
<code>application.properties</code>	Configure server port, database, logging, profiles, and more
Starters	Pre-bundled compatible dependencies (one line = many libraries)
Auto-configuration	Spring Boot configures itself based on your dependencies
Embedded Tomcat	Web server built into your JAR — no external server needed



What to take away from this lesson

1. Spring Boot = Spring Framework made easy. Auto-configuration, embedded server, starter dependencies.
2. Every project starts at <https://start.spring.io> — generates a ready-to-run project.
3. Three layers: Controller (HTTP) → Service (logic) → Repository (database), connected by DI.
4. Annotations tell Spring what each class does: `@RestController` , `@Service` , `@Repository` , `@Entity` .
5. REST annotations map HTTP methods to Java methods: `@GetMapping` ,

`@PostMapping` , `@DeleteMapping` .

6. Use `ResponseBody` for custom status codes and `@RestControllerAdvice` for global error handling.

7. Profiles let you switch between dev/prod config without changing code.

8. Lombok exists but we'll skip it for now — write getters/setters manually to understand the fundamentals first. Actuator adds health checks.

`@SpringBootTest` makes testing easy.

9. `application.properties` configures everything: port, database, logging — one file, no XML.

10. Each microservice is a separate Spring Boot app with its own port, database, and configuration.

The complete learning path: Microservices (architecture) → **REST API** (communication) → **Spring Framework** (IoC, DI, Beans) → **Spring Ecosystem** (the map) → **Spring Boot** (today — building microservices). You now have the full conceptual foundation to start building real projects.